

CompSci Graph Theory Notes

Jesse Oldroyd

July 1, 2026

2026-07-01

CompSci Graph Theory Notes

CompSci Graph Theory Notes

Jesse Oldroyd
July 1, 2026

Computer Science and Graph Theory

2026-07-01

CompSci Graph Theory Notes
└ Computer Science and Graph Theory

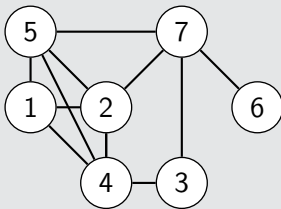
Computer Science and Graph
Theory

Computer Science and Graph Theory

- Graphs can be represented using **adjacency matrices**
- Adjacency matrices are rectangular arrays that use 1 to indicate edges in a graph

Example (Finding an adjacency matrix)

Find the adjacency matrix for the following graph:



- Graphs can be represented using **adjacency matrices**
- Adjacency matrices are rectangular arrays that use 1 to indicate edges in a graph

Example (Finding an adjacency matrix)

Find the adjacency matrix for the following graph:



- Matrices can be handled in Python using the NumPy arrays:

```
import numpy as np

A = np.array([
    [0, 1, 0, 1, 1, 0, 0],
    [1, 0, 0, 1, 1, 0, 1],
    [0, 0, 0, 1, 0, 0, 1],
    [1, 1, 1, 0, 1, 0, 0],
    [1, 1, 0, 1, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 1],
    [0, 1, 1, 0, 1, 1, 0]
])
```

2026-07-01

CompSci Graph Theory Notes

- Computer Science and Graph Theory
 - Matrices and Python

• Matrices can be handled in Python using the NumPy arrays:

```
import numpy as np

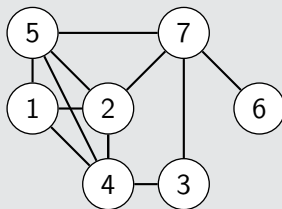
A = np.array([
    [0, 1, 0, 1, 1, 0, 0],
    [1, 0, 0, 1, 1, 0, 1],
    [0, 0, 0, 1, 0, 0, 1],
    [1, 1, 1, 0, 1, 0, 0],
    [1, 1, 0, 1, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 1],
    [0, 1, 1, 0, 1, 1, 0]
])
```

- NumPy provides many operations to use with arrays
 1. Column/row sums with `sum` (*not* `np.sum`). This is useful for finding degrees of vertices.
 2. Matrix multiplication with `@` (such as `A@A`). This is useful for counting paths between vertices.

Example (Adjacency matrices, paths and degrees)

Use the adjacency matrix to answer the following questions:

1. What is the degree of each vertex?
2. How many paths of length two are there between vertex 4 and vertex 7?
3. How many cycles of length four start at vertex 2?



2026-07-01



- We can draw graphs using NetworkX and matplotlib.

```
import matplotlib.pyplot as plt
import networkx as nx

G = nx.from_numpy_array(A)
nx.display(G)
plt.show()
```

- We can draw graphs using NetworkX and matplotlib.

```
import matplotlib.pyplot as plt
import networkx as nx

G = nx.from_numpy_array(A)

nx.display(G)
plt.show()
```

This code should be continued from previous code!

Example (Utility graph)

Plot the utility graph using Python.

2026-07-01

CompSci Graph Theory Notes
└ Computer Science and Graph Theory
└ Drawing Graphs Again

Example (Utility graph)

Plot the utility graph using Python.

Using the algorithm in Python

Using the provided algorithm `gale_shapley_algorithm.py`, solve the stable matching problem for the following set of preferences:

Applicant	Employer rankings	Employer	Applicant rankings
1	B, A, C	A	3, 1, 2
2	B, C, A	B	2, 3, 1
3	A, B, C	C	1, 3, 2

Using the algorithm in Python

Using the provided algorithm `gale_shapley_algorithm.py`, solve the stable matching problem for the following set of preferences:

Applicant	Employer rankings	Employer	Applicant rankings
1	B, A, C	A	3, 1, 2
2	B, C, A	B	2, 3, 1
3	A, B, C	C	1, 3, 2

- We can also use **dictionaries** in Python to build graphs with NetworkX.
- Dictionaries in python associate a **key** with a **value**:
`{key1: value1, key2: value2, ...}`.

Example (Utility graph from dictionary)

Use a dictionary to create the utility graph.

2026-07-01

- We can also use **dictionaries** in Python to build graphs with NetworkX.
- Dictionaries in python associate a **key** with a **value**:
`{key1: value1, key2: value2, ...}`.

Example (Utility graph from dictionary)

Use a dictionary to create the utility graph.

BFS algorithm:

1. Add the root to the queue. While the queue is nonempty, do the following:
2. Set the current vertex equal to the first vertex in the queue, removing it from the queue.
3. If the current vertex is the target, then set the current vertex as the parent of the target and erase the queue.
4. If the current vertex is not the target, then do the following for each neighbor:
 - 4.1 set the parent of the neighbor to the current vertex.
 - 4.2 add the neighbor to the queue.

Example (Solving a maze)

Implement the breadth-first search algorithm to solve the maze in the file `maze.py`.

2026-07-01

CompSci Graph Theory Notes

- └ Computer Science and Graph Theory
 - └ BFS in Python

BFS algorithm:

1. Add the root to the queue. While the queue is nonempty, do the following:
2. Set the current vertex equal to the first vertex in the queue, removing it from the queue.
3. If the current vertex is the target, then set the current vertex as the parent of the target and erase the queue.
4. If the current vertex is not the target, then do the following for each neighbor:
 - 4.1 set the parent of the neighbor to the current vertex.
 - 4.2 add the neighbor to the queue.

Example (Solving a maze)

Implement the breadth-first search algorithm to solve the maze in the file `maze.py`.